VendoRL: Smart Pricing for a Vending Machine

Artificial Intelligence Assignment II

Shiv Jee Yadav (B23EE1095)
Ritik Nagar (B23EE1086)

IIT Jodhpur
GitHub: https://github.com/ritiknagar9521/VendoRL

---

1 What Is the Project About?

Imagine running a vending machine for one day. If the prices are too low, snacks and drinks may sell out early, reducing the chance to earn more. If prices are too high, customers may buy less and some products may remain unsold.

Our project, VendoRL, explores whether a computer agent can learn better prices by trying different choices in a simulated vending machine.

The machine sells one snack and one drink during a 12-hour shift. Customer demand changes from day to day. The agent does not know whether the day has low, normal, or high demand. It must use recent sales and the current situation to make pricing decisions.

2 How the Vending Machine Works

We created the environment in Python. At the beginning of each shift, the machine has 30 snacks and 30 drinks. Each hour, the agent chooses prices, customers arrive randomly, sales are recorded, and the remaining stock is updated.

- **Snack prices:** Rs 20, Rs 40, or Rs 70.
- **Drink prices:** Rs 30, Rs 50, or Rs 90.
- **Shift duration:** 12 hours, starting at 08:00.
- **Lunch rush:** Demand increases from 12:00 to 14:00.
- **Competitor:** A cafeteria is open from 12:00 to 16:00, making customers more sensitive to prices.
- **Demand levels:** Low, normal, and high, with probabilities of 30%, 40%, and 30%.

Customer arrivals are generated randomly using a Poisson model. Higher prices generally reduce expected demand, while the lunch rush and high-demand days increase it. Sales cannot exceed the available stock.

This setup gives the agent a simple way to learn how prices affect sales and profit under uncertain customer demand.

3 What Does the Agent Observe?

At each hour, the agent receives eight numbers as its observation. These values are scaled to make them easier for the neural network to process.

- Remaining snack stock and drink stock.
- Current time in the shift.
- Whether the cafeteria competitor is open.

- Snack and drink sales during the previous hour.
- Previous snack and drink price levels.

The actual demand level is hidden from the agent. It cannot directly see whether the day is low-, normal-, or high-demand. Instead, it can use recent sales as clues.

The agent has nine possible actions. Each action is one combination of the three snack prices and three drink prices.

4 How Is Profit Calculated?

The main goal is to earn as much profit as possible over the full shift, rather than simply selling the largest number of products.

The reward for each hour includes the money earned from sold products after accounting for their purchase costs. It also subtracts the cost of keeping unsold stock. At the end of the shift, the purchase cost of any remaining products is subtracted.

In simple terms, the reward considers:

- Profit earned from each snack and drink sold.
- A holding cost of Rs 1 per remaining unit per hour.
- The purchase cost of products left unsold at closing time.

A snack costs Rs 15 and a drink costs Rs 20. This reward encourages the agent to balance higher selling prices against the risk of having too much stock left at the end of the day.

5 Why Did We Use PPO?

We used Proximal Policy Optimization (PPO), a reinforcement learning algorithm that improves its decisions through repeated interaction with the environment.

Our implementation uses two small neural networks:

- **Actor:** Chooses the probability of each of the nine possible price combinations.
- **Critic:** Estimates how much reward the agent may receive from its current situation.

During training, the agent tries different prices and receives rewards based on the results. PPO uses this experience to improve its policy gradually. Its update method limits how much the policy changes at once, helping training remain stable.

The main training settings are:

- Learning rate: 0.0003.
- Discount factor: 0.99.
- Eight update epochs per training batch.
- 32 episodes collected in each training batch.
- Three random seeds to compare different training runs.

6 Results and Comparison

We compared PPO with three simple strategies:

- **Random pricing:** Selects price combinations randomly.
- **Best fixed price:** Always uses Rs 40 for a snack and Rs 50 for a drink.
- **Time-based rule:** Uses cheaper prices while the cafeteria is open and normal prices at other times.

The following dashboard is the only results image available for this project. It combines the learning curve, profit comparisons, unsold stock, hourly prices, and profit for different demand levels.

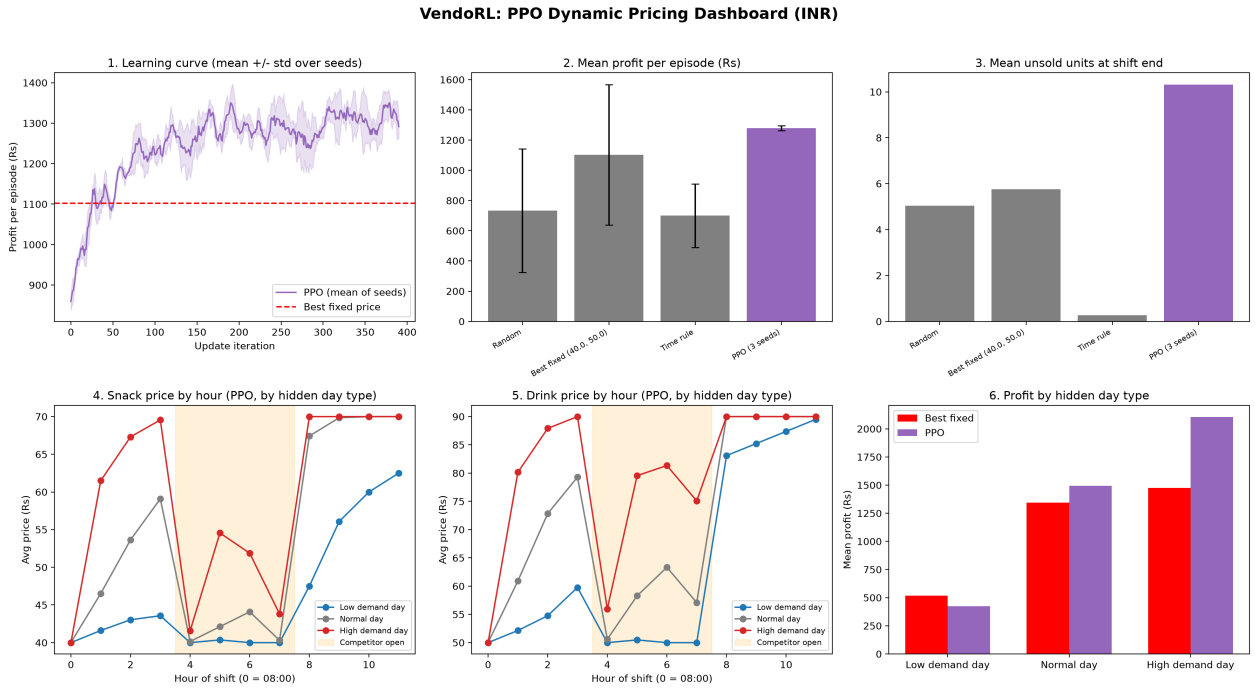


Figure 1: VendoRL results dashboard. The plots compare PPO with baseline strategies and show how prices and profit change with time and demand level.

Based on approximate readings from the dashboard, the mean profits per shift are:

Strategy	**Profit (Rs)**	**Unsold units**
Random pricing	≈ 730	≈ 5.0
Time-based rule	≈ 700	≈ 0.3
Best fixed price	≈ 1100	≈ 5.8
PPO	≈ 1280	≈ 10.3

These figures are approximate readings from the supplied dashboard, not freshly calculated results from the evaluation script.

7 Results Inference

The learning curve rises during early training and later stays mostly around Rs 1,250–1,330 per episode. The dashboard suggests that PPO earns about Rs 1280 per shift, compared with approximately Rs 1100 for the best fixed-price strategy. This is an improvement of roughly 16% based on the displayed values.

The results also show a few important points:

- On low-demand days, PPO earns less than the fixed-price strategy.
- On normal-demand days, PPO performs somewhat better than the fixed-price strategy.
- On high-demand days, PPO shows its largest profit advantage.
- PPO leaves more products unsold on average than the fixed-price strategy.

The hourly price plots suggest that the agent changes its prices during the shift. Prices generally fall around the competitor's opening period and rise again after it closes. The agent also tends to choose higher prices on high-demand days.

This behaviour is consistent with the agent using recent sales to estimate demand, although the dashboard alone cannot prove exactly what the neural network has learned.

8 Limitations and Future Improvements

This project uses a simulation, so its results may differ from what happens in a real vending machine. The demand settings were chosen for the project and have not been calibrated using real customer data.

PPO also does not perform best in every demand category, and its higher average profit comes with more unsold stock.

If we continue this project, we would like to:

- Test more random seeds and evaluation episodes.
- Use real sales data to improve the demand model.
- Try more price levels and add restocking decisions.
- Explore an agent with memory to estimate demand more reliably from recent sales.

9 Conclusion

VendoRL shows how reinforcement learning can help with an everyday pricing problem. Instead of always using the same prices, PPO learns from simulated shifts and adjusts prices according to stock, time, competition, and recent sales.

In the supplied dashboard, PPO has the highest overall mean profit among the compared strategies. However, it also leaves more stock unsold and performs less well on low-demand days. Testing the policy with real customer demand data would be an important next step.